Übungsblatt 3: Speicherzugriff und Kontrollfluss

Mikrocomputertechnik 1

Musterlösung

Aufgabe 1: Die Messreihe im Speicher (10 Punkte)

Ein Frostwächter überwacht nachts die Temperatur in einem Gewächshaus; er begleitet Sie durch das ganze Blatt. Stündlich legt er die Temperatur in ganzen Grad Celsius als signed 32-Bit-Wort (Zweierkomplement) im Array T ab. Die Messreihe der letzten Nacht lautet T = [−5, 8, −2, 7, −1, 0]. Direkt dahinter liegt ein Ergebniswort für die spätere Anzahl der Frostwerte; −1 bedeutet „noch kein Ergebnis“. Lehrannahme: T beginnt bei B = `0x2000`, der Speicher ist Little Endian, alle Zugriffe sind gültig. s3 = `0x2000` hält die feste Basis, s1 = `0x2008` zeigt als laufender Pointer auf T[2].



1. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. (3 P)

Index i	Adresse B + 4i	Wert	Wort hexadezimal	Bytes ab der Adresse
0	`0x2000`	-5	`0xFFFFFFFB`	`FB FF FF FF`
2				
3				
	`0x2010`			

2. Die Befehle laufen nacheinander. Ergänzen Sie die effektive Adresse und den Registerwert danach. Die erste Zeile ist ein Beispiel. (4 P)

Befehl	effektive Adresse	Registerwert danach
`lw t0, 0(s1)`	`0x2008`	t0 = −2 = `0xFFFFFFFE`
`lw t1, -8(s1)`		
`lw t2, 12(s1)`		
`addi t3, s1, 4`		
`lw t4, 0(t3)`		

3. Ein Kollege will T[3] über die feste Basis lesen und schreibt `lw t5, 3(s3)`. Welche effektive Adresse entsteht? Ist sie für ein Wort natürlich ausgerichtet? Welcher Offset wäre richtig? (1 P)
4. Der Wert T[3] = 7 war ein Messfehler. Das Programm korrigiert ihn mit s1 = `0x2008`:

```
lw   t0, 4(s1)
addi t0, t0, -10
sw   t0, 4(s1)
```

An welche Adresse schreibt der Store, und welche vier Bytes stehen danach an den aufsteigenden Adressen? Welchen Wert hat s1 danach? Wie viele Werte von T sind jetzt negativ? (2 P)

Für alle weiteren Aufgaben gilt wieder die ursprüngliche Messreihe.

Musterlösung

Index i	Adresse B + 4i	Wert	Wort hexadezimal	Bytes ab der Adresse
2	`0x2008`	−2	`0xFFFFFFFE`	`FE FF FF FF`
3	`0x200C`	7	`0x00000007`	`07 00 00 00`
4	`0x2010`	−1	`0xFFFFFFFF`	`FF FF FF FF`

1. Je Zeile 1 P, anteilig.

Befehl	effektive Adresse	Registerwert danach
`lw t1, -8(s1)`	`0x2000`	t1 = −5 = `0xFFFFFFFB`
`lw t2, 12(s1)`	`0x2014`	t2 = 0
`addi t3, s1, 4`	kein Speicherzugriff	t3 = `0x200C`, eine Adresse
`lw t4, 0(t3)`	`0x200C`	t4 = 7

2. Je Zeile 1 P. s1 bleibt bei allen Befehlen `0x2008`; der Offset wird nur für den Zugriff addiert. addi kopiert eine Adresse, erst lw liest den Inhalt.
3. `0x2003`. Bei Division durch vier bleibt der Rest 3, die Adresse ist nicht ausgerichtet (0,5 P). Der Index zählt Wörter, der Offset Bytes: richtig ist $4 \cdot 3 = 12$, also `lw t5, 12(s3)` (0,5 P).
4. Der Store schreibt an `0x200C` (0,5 P). $7 - 10 = -3$ = `0xFFFFFFFD`, die Bytes lauten `FD FF FF FF` (0,5 P). s1 bleibt `0x2008` (0,5 P). T = [−5, 8, −2, −3, −1, 0] enthält vier negative Werte (0,5 P).

Aufgabe 2: Teilzugriffe auf das Statuswort (10 Punkte)

Zusätzlich legt der Frostwächter ein Statuswort ab. s2 = `0x2100` zeigt auf seine vier Bytes:

Adresse	`0x2100`	`0x2101`	`0x2102`	`0x2103`
Byte	`F6`	`3C`	`06`	`FF`
Bedeutung	Bodentemperatur	Luftfeuchte	Lufttemperatur, unteres Byte	Lufttemperatur, oberes Byte

Die Bodentemperatur in °C ist ein signed Byte, die Luftfeuchte in % ein vorzeichenloses Byte. Die Lufttemperatur ist ein signed Halbwort in Hundertstel Grad: 125 bedeutet 1,25 °C.

1. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. (4 P)

Befehl	Registerwort	Zahlenwert
`lbu t0, 1(s2)`	`0x0000003C`	60
`lb t1, 0(s2)`		
`lbu t2, 0(s2)`		
`lh t3, 2(s2)`		
`lhu t4, 2(s2)`		

2. Welcher Load passt zur Bodentemperatur, welcher zur Luftfeuchte, welcher zur Lufttemperatur? Eine Anzeige meldet für die Luft 652,86 °C. Welchen Load hat sie verwendet, und welche Temperatur ist gemeint? (2 P)
3. Danach gilt t5 = 55 = `0x00000037` und t6 = −200 = `0xFFFFFF38`. Nacheinander laufen `sb t5, 1(s2)` und `sh t6, 2(s2)`. Ergänzen Sie die Bytes. (2 P)

Zeitpunkt	`0x2100`	`0x2101`	`0x2102`	`0x2103`
vorher	`F6`	`3C`	`06`	`FF`
nach `sb t5, 1(s2)`				
nach `sh t6, 2(s2)`				

4. Welches Wort liefert danach `lw t0, 0(s2)`? Welche Lufttemperatur liefert `lh t3, 2(s2)`? (1 P)
5. Angenommen, t6 enthält `0x0000FF38` statt `0xFFFFFF38`. Welche Bytes schreibt `sh t6, 2(s2)` dann? Warum braucht RV32I deshalb keinen eigenen unsigned Store wie „shu“? (1 P)

Musterlösung

Befehl	Registerwort	Zahlenwert
`lb t1, 0(s2)`	`0xFFFFFFF6`	−10
`lbu t2, 0(s2)`	`0x000000F6`	246
`lh t3, 2(s2)`	`0xFFFFFF06`	−250
`lhu t4, 2(s2)`	`0x0000FF06`	65 286

1. Je Zeile 1 P. Little Endian: Die Bytes `06 FF` ergeben das Halbwort `0xFF06`, Bit 15 ist eins.
2. Bodentemperatur lb, Luftfeuchte lbu, Lufttemperatur lh (1 P). Die Anzeige hat lhu verwendet: $65\,286$ Hundertstel ergeben 652,86 °C. Gemeint sind −250 Hundertstel, also −2,50 °C (1 P).

Zeitpunkt	`0x2100`	`0x2101`	`0x2102`	`0x2103`
nach `sb t5, 1(s2)`	`F6`	`37`	`06`	`FF`
nach `sh t6, 2(s2)`	`F6`	`37`	`38`	`FF`

3. Je Zeile 1 P. sb ersetzt nur das Byte an `0x2101`, sh nur die zwei Bytes ab `0x2102` mit den unteren 16 Bit `0xFF38` in Little-Endian-Folge.
4. `0xFF3837F6` (0,5 P). lh liefert `0xFFFFFF38` = −200, also −2,00 °C (0,5 P).
5. Ebenfalls `38 FF`, denn sh schreibt nur die unteren 16 Bit. Ein Store verkleinert die Breite und erweitert nichts; erst der spätere Load entscheidet über Vorzeichen- oder Nullerweiterung (1 P).

Aufgabe 3: Stores im Maschinenwort (8 Punkte)

Store-Befehle verwenden das S-Format. Die Felder stehen von Bit 31 bis Bit 0 in dieser Reihenfolge:

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
7 Bit	5 Bit	5 Bit	3 Bit	5 Bit	7 Bit

Für sw gilt funct3 = `010` und opcode = `0100011`. Registernummern: t0 = x5, s0 = x8, s1 = x9, s3 = x19. Die erste Zeile jeder Tabelle ist das Beispiel aus der Vorlesung.

1. Bilden Sie den zwölfbittigen Offset und teilen Sie ihn auf. Die zweite Zeile ist der Korrektur-Store aus Aufgabe 1. Tipp: Ein negativer Offset wird als zwölfbittiges Zweierkomplement geschrieben, etwa $2^{12} - 4 = 4092$ = `0xFFC`. (2 P)

Befehl	Offset	12-Bit-Muster	imm[11:5]	imm[4:0]
`sw t0, -4(s1)`	-4	`1111 1111 1100`	`1111111`	`11100`
`sw t0, 4(s1)`	4			
`sw t0, -8(s1)`	-8			

2. Setzen Sie für dieselben drei Stores die Felder zusammen und geben Sie das Hexwort an. Tipp: Gruppieren Sie die 32 Bit von links in Vierergruppen. (2 P)

Offset	imm[11:5]	rs2	rs1	f3	imm[4:0]	opcode	Hexwort
-4	`1111111`	`00101`	`01001`	`010`	`11100`	`0100011`	`0xFE54AE23`
4							
-8							

3. Am Ende legt das Programm des Frostwächters die Anzahl der Frostwerte mit einem Store ab. Sein Maschinenwort ist `0x0089AC23`. Zerlegen Sie es in die sechs Felder, setzen Sie den Offset zusammen und geben Sie den Befehl mit Aliasnamen an. Tipp: Schreiben Sie zuerst jede Hexziffer als vier Bit. (3 P)
4. Die Bits 11:7 dieses Worts lauten `11000`. Warum wird trotzdem kein Register x24 beschrieben? (1 P)

Musterlösung

Befehl	Offset	12-Bit-Muster	imm[11:5]	imm[4:0]
`sw t0, 4(s1)`	4	`0000 0000 0100`	`0000000`	`00100`
`sw t0, -8(s1)`	-8	`1111 1111 1000`	`1111111`	`11000`

1. Je Zeile 1 P. Für -8 gilt $2^{12} - 8 = 4088$ = `0xFF8`.

Offset	imm[11:5]	rs2	rs1	f3	imm[4:0]	opcode	Hexwort
4	`0000000`	`00101`	`01001`	`010`	`00100`	`0100011`	`0x0054A223`
-8	`1111111`	`00101`	`01001`	`010`	`11000`	`0100011`	`0xFE54AC23`

2. Je Zeile 1 P (Felder 0,5 P, Hexwort 0,5 P). Vierergruppen: `0000 0000 0101 0100 1010 0010 0010 0011` bzw. `1111 1110 0101 0100 1010 1100 0010 0011`.
3. `0000 0000 1000 1001 1010 1100 0010 0011` ergibt die Felder `0000000 01000 10011 010 11000 0100011` (1 P). Opcode und funct3 bedeuten sw; rs2 = 8 = s0 liefert die Daten, rs1 = 19 = s3 die Basis (1 P). Der Offset `0000000` + `11000` = `000000011000` ist +24; der Befehl lautet `sw s0, 24(s3)` (1 P).
4. Das S-Format hat kein rd. Die Bits 11:7 sind imm[4:0], der untere Teil des Offsets 24. Der Store schreibt nur in den Speicher (1 P).

Aufgabe 4: Die Frostanzeige (12 Punkte)

Nach jeder Messung zeigt der Frostwächter einen Code an: 1 bedeutet OK, 2 bedeutet Frost. t2 enthält den aktuellen Messwert, s4 die Adresse `0x2200` des Anzeigefelds. Der erste Befehl liegt bei `0x100`. Jeder Befehl belegt vier Byte; `j Anzeige` ist hier genau ein nativer Befehl (`jal x0, Anzeige`).

```
          blt  t2, x0, Frost      # Messwert negativ?
          addi t5, x0, 1          # Code 1: OK
          j    Anzeige
Frost:    addi t5, x0, 2          # Code 2: Frost
Anzeige:  sw   t5, 0(s4)          # Code ins Anzeigefeld
```

1. Ergänzen Sie die Adressen. Welche Adressen benennen die Labels Frost und Anzeige? (2 P)

Befehl	Adresse
`blt t2, x0, Frost`	`0x100`
`addi t5, x0, 1`	
`j Anzeige`	
`addi t5, x0, 2`	
`sw t5, 0(s4)`	

2. Bestimmen Sie die Byteoffsets von blt und j. Tipp: Offset = Zieladresse − Adresse des Sprungbefehls. (2 P)
3. Verfolgen Sie das Programm. Die erste Zeile ist ein Beispiel. (2 P)

t2	t2 als Wort	blt genommen?	ausgeführte Adressen (hex)	Code
8	`0x00000008`	nein	100 → 104 → 108 → 110	1
−5				
0				

4. In einer Version zeigt die Anzeige auch bei −5 °C den Code 1. Dort steht `bltu t2, x0, Frost` statt `blt t2, x0, Frost`. Als welche Zahl liest bltu das Wort von −5? Gibt es einen Messwert, bei dem diese Version Frost anzeigt? (2 P)
5. In einer anderen Version zeigt die Anzeige auch bei 8 °C den Code 2. Dort fehlt die Zeile `j Anzeige`. Geben Sie die neuen Adressen von Frost und Anzeige sowie die ausgeführten Adressen für t2 = 8 an. Wo wird der Code überschrieben? (2 P)
6. Künftig soll schon 0 °C als Frost gelten, also t2 ≤ 0. Ersetzen Sie nur den Branch durch einen nativen Befehl, indem Sie die Quellen vertauschen. Welcher Code erscheint dann für t2 = 0 und für t2 = 8? (2 P)

Musterlösung

Befehl	Adresse
`addi t5, x0, 1`	`0x104`
`j Anzeige`	`0x108`
`addi t5, x0, 2`	`0x10C`
`sw t5, 0(s4)`	`0x110`

1. Je Zeile 0,5 P. Frost = `0x10C`, Anzeige = `0x110`; Labels belegen keinen Platz.
2. blt: $\texttt{0x10C} - \texttt{0x100} = +12$ (1 P). j: $\texttt{0x110} - \texttt{0x108} = +8$ (1 P).

t2	t2 als Wort	blt genommen?	ausgeführte Adressen (hex)	Code
−5	`0xFFFFFFFB`	ja	100 → 10C → 110	2
0	`0x00000000`	nein	100 → 104 → 108 → 110	1

3. Je Zeile 1 P. 0 ist nicht kleiner als 0, die Anzeige meldet OK.
4. bltu liest `0xFFFFFFFB` als $4\,294\,967\,291$; das ist nicht kleiner als 0 (1 P). Kein vorzeichenloser Wert ist kleiner als 0, diese Version zeigt nie Frost (1 P).
5. Frost = `0x108`, Anzeige = `0x10C` (1 P). Für t2 = 8: 100 → 104 → 108 → 10C. Der Code 1 aus `0x104` wird bei `0x108` durch 2 überschrieben, weil die Ausführung ohne Sprung in den Frost-Block weiterläuft (1 P).
6. `bge x0, t2, Frost` springt bei $0 \geq t2$, also bei $t2 \leq 0$ (1 P). t2 = 0: Code 2; t2 = 8: Code 1 (1 P).

Aufgabe 5: Frostwerte zählen (10 Punkte)

Der Frostwächter zählt die negativen Werte der Messreihe T aus Aufgabe 1. Die Schleife folgt dem Registerplan der Vorlesung: t0 = i (verarbeitete Elemente), t1 = N, s1 = laufender Pointer, s0 = Anzahl der Frostwerte, t2 = zuletzt geladener Wert. Vorher gilt s1 = B = `0x2000`; t2 ist unbekannt.

```
    addi t0, x0, 0        # i = 0
    addi t1, x0, 6        # N = 6
    addi s0, x0, 0        # Anzahl = 0
Loop:
    bge  t0, t1, Done     # alle N Werte verarbeitet?
    lw   t2, 0(s1)
    bge  t2, x0, Next     # nicht negativ: nicht zählen
    addi s0, s0, 1
Next:
    addi s1, s1, 4
    addi t0, t0, 1
    j    Loop
Done:
```

1. Notieren Sie den Zustand am Label Loop vor jedem `bge t0, t1, Done`, einschließlich des letzten Tests. Die erste Zeile ist ein Beispiel. Tipp: Am Label Loop gilt immer s1 = B + 4 · t0. (6 P)

Test	t0	s1	s0	t2	Sprung nach Done?
1	0	`0x2000`	0	unbekannt	nein
2					
3					
4					
5					
6					
7					

2. Wie oft werden `lw` und `addi s0, s0, 1` ausgeführt, wie oft der Test bei Loop? Auf welche Adresse zeigt s1 am Ende, und warum wird von dort nicht mehr geladen? (2 P)
3. Wiederholen Sie gedanklich mit N = 0 und mit N = 2. Geben Sie jeweils s0, s1 am Ende und die Zahl der Loads an. (2 P)

Musterlösung

Test	t0	s1	s0	t2	Sprung nach Done?
2	1	`0x2004`	1	−5	nein
3	2	`0x2008`	1	8	nein
4	3	`0x200C`	2	−2	nein
5	4	`0x2010`	2	7	nein
6	5	`0x2014`	3	−1	nein
7	6	`0x2018`	3	0	ja

1. Je Zeile 1 P.
2. Sechs Loads, drei Zählschritte und sieben Tests (1 P). s1 = `0x2018` = B + 24 zeigt auf das Ergebniswort hinter T. Der siebte Test springt mit t0 = 6 nach Done, bevor ein weiterer Load stattfindet (1 P).
3. N = 0: s0 = 0, s1 = `0x2000`, kein Load; der erste Test springt sofort, t2 bleibt unbekannt (1 P). N = 2: s0 = 1, s1 = `0x2008`, zwei Loads (1 P).

Aufgabe 6: Fehlersuche und Ausbau (10 Punkte)

Drei Teams haben die Zählschleife aus Aufgabe 5 abgetippt und mit derselben Messreihe gestartet. Team A und Team B erhalten mit N = 6 beide s0 = 6 statt 3. Beiden fehlt genau eine Zeile. Endzustände bei Done:

Team	t0	s1	t2	s0
A	6	`0x2000`	−5	6
B	6	`0x2018`	0	6

1. Welche Werte hat Team A nacheinander geladen? Welche Zeile fehlt? (2 P)
2. Team B hat alle sechs Werte geladen. Welche Zeile fehlt dort? Begründen Sie mit s0. (2 P)
3. Team C setzt den Test ans Ende der Schleife, die drei Initialisierungen bleiben gleich. Für N = 6 stimmt das Ergebnis 3. Bestimmen Sie für N = 0 das Ergebnis und die Zahl der Loads. Was ist an dieser Form falsch? (2 P)

```
Loop:
    lw   t2, 0(s1)
    bge  t2, x0, Next
    addi s0, s0, 1
Next:
    addi s1, s1, 4
    addi t0, t0, 1
    blt  t0, t1, Loop
Done:
```

4. Nach Done soll die Anzahl ins Ergebniswort bei `0x2018` (anfangs −1). Ein Kollege schreibt `sw s0, 0(s1)`. Für N = 6 stimmt das. Was steht bei N = 4 danach in T[4] und im Ergebniswort? Welcher Befehl aus Aufgabe 3 arbeitet unabhängig von N richtig, wenn s3 wie in Aufgabe 1 die feste Basis B enthält? (2 P)
5. Ausbau: Nur starker Frost unter −1 °C soll zählen. Vor Loop setzt `addi t3, x0, -1` den Grenzwert. Ändern Sie genau einen Befehl im Rumpf. Welche Anzahl ergibt sich für T? Wird der Wert −1 gezählt? (2 P)

Musterlösung

1. Sechsmal −5 von `0x2000`, denn s1 bleibt stehen (1 P). Es fehlt `addi s1, s1, 4` (1 P).
2. Es fehlt `bge t2, x0, Next` (1 P). Pointer und t2 enden wie im korrekten Lauf, aber jeder Durchlauf erreicht `addi s0, s0, 1`; damit wird s0 = N = 6 (1 P).
3. N = 0: Der Rumpf läuft vor dem ersten Test. Er lädt T[0] = −5 und setzt s0 = 1; danach ist $1 < 0$ falsch. Ergebnis 1 statt 0 nach einem Load (1 P). Bei N = 0 darf kein Load stattfinden; nur ein Test vor dem Rumpf schützt den Zugriff (1 P).
4. Bei N = 4 ist s1 = `0x2010` und s0 = 2 (−5 und −2). Der Store überschreibt T[4] = −1 mit 2, das Ergebniswort bleibt −1 (1 P). Richtig ist `sw s0, 24(s3)` mit der festen Basis (1 P).
5. `bge t2, t3, Next` statt `bge t2, x0, Next` (1 P). Gezählt werden −5 und −2, die Anzahl ist 2. −1 zählt nicht, denn bei $-1 \geq -1$ springt der Branch nach Next (1 P).

Aufgabe 7: Die Zählschleife in Venus prüfen (optional)

Das Programm führt die Zählschleife aus Aufgabe 5 mit dem Ergebnis-Store aus Aufgabe 3 aus. `la` lädt die tatsächliche Adresse B der Messreihe nach s1, `addi s3, s1, 0` kopiert sie als feste Basis. Danach gibt der Venus-Dienst 1 die Anzahl aus, Dienst 10 beendet das Programm.

```
.data
messwerte: .word -5, 8, -2, 7, -1, 0
ergebnis:  .word -1              # noch kein Ergebnis
.text
.globl main
main:
    la   s1, messwerte           # laufender Pointer
    addi s3, s1, 0               # feste Basis B
    addi t0, x0, 0               # i = 0
    addi t1, x0, 6               # N = 6
    addi s0, x0, 0               # Anzahl = 0
Loop:
    bge  t0, t1, Done
    lw   t2, 0(s1)
    bge  t2, x0, Next
    addi s0, s0, 1
Next:
    addi s1, s1, 4
    addi t0, t0, 1
    j    Loop
Done:
    sw   s0, 24(s3)              # Anzahl ins Ergebniswort
Ergebnis_pruefen:
    mv   a1, s0
    li   a0, 1                  # Dienst 1: Ganzzahl ausgeben
    ecall
    li   a0, 10                  # Dienst 10: Programmende
    ecall
```

1. Öffnen Sie die Kurs-Venus unter thaumicmekanism.github.io/venus. Geben Sie das Programm im Tab *Editor* ein; ein Klick auf den Tab *Simulator* assembliert es. Die Programmansicht zeigt je Zeile das Maschinenwort, den nativen Befehl nach Auflösung der Pseudoinstruktionen und Ihren Quelltext. Wie viele native Befehle erzeugt `la`? Stimmt das Maschinenwort von `sw s0, 24(s3)` mit Aufgabe 3 überein?
2. Führen Sie das Programm mit *Step* Befehl für Befehl aus; die grün markierte Zeile wird als Nächstes ausgeführt, *Prev* nimmt den letzten Schritt zurück. Lesen Sie nach `la` die Basis B in s1 ab. Werte erscheinen standardmäßig vorzeichenlos hexadezimal (*Hex*), die Anzeigeeinstellung *Decimal* zeigt sie signed dezimal.
3. Vergleichen Sie vor jedem `bge t0, t1, Done` die Register t0, s1, s0 und t2 mit Ihrer Tabelle aus Aufgabe 5; ersetzen Sie dabei `0x2000` durch B. Tipp: Ein Klick auf einen Befehl setzt einen Haltepunkt (rote Zeile), *Run* läuft bis zum nächsten Haltepunkt.
4. Prüfen Sie bei `Ergebnis_pruefen` s0 und im Tab *Memory* (Adresse eingeben oder über *Jump to* zum Datenbereich) die vier Bytes des Ergebnisworts bei B + 24; vergleichen Sie mit dem Zustand vor dem Store. Die Konsole zeigt die ausgegebene Anzahl.
5. Ändern Sie das Programm im Editor, wechseln Sie erneut zum Tab *Simulator* und starten Sie neu: N = 0 und N = 1; die Messwerte 0, 2, 9, 4, −3, −8 mit N = 4 und N = 6; der Grenzwert aus Aufgabe 6.5. Optional: `sw s0, 0(s1)` statt `sw s0, 24(s3)` mit N = 4, um Ihre Vorhersage aus Aufgabe 6.4 im Speicher zu prüfen.
6. Protokollieren Sie je Lauf B, die Ausgabe, die Zahl der Loads (gleich t0 am Ende), den Pointerabstand s1 − B bei `Ergebnis_pruefen` und die Bytes des Ergebnisworts. Notieren Sie Vorhersage und Beobachtung in getrennten Spalten.

Musterlösung

Erwartung bei `Ergebnis_pruefen` für die vorgegebene Messreihe: s0 = 3, sechs Loads, s1 − B = 24 Byte, Ergebniswort `03 00 00 00` statt vorher `FF FF FF FF`. Das Maschinenwort des Stores ist `0x0089AC23` wie in Aufgabe 3. `la` entspricht nach der üblichen Standardentwicklung zwei nativen Befehlen (`auipc` und `addi`); maßgeblich ist die Anzeige. Laut Venus-Dokumentation beginnt der statische Datenbereich bei `0x10000000`; dann gilt B = `0x10000000`, und das Ergebniswort liegt bei `0x10000018`. Auch hier zählt die eigene Anzeige.

Lauf	Ausgabe	Loads	s1 − B	Ergebniswort danach
Messreihe, N = 6	3	6	24	`03 00 00 00`
Messreihe, N = 0	0	0	0	`00 00 00 00`
Messreihe, N = 1	1	1	4	`01 00 00 00`
0, 2, 9, 4, −3, −8, N = 4	0	4	16	`00 00 00 00`
0, 2, 9, 4, −3, −8, N = 6	2	6	24	`02 00 00 00`
Grenzwert −1, N = 6	2	6	24	`02 00 00 00`
`sw s0, 0(s1)`, N = 4	2	4	16	`FF FF FF FF`, unverändert

Bei N = 4 zählt die Schleife nur die ersten vier Werte; −3 und −8 dahinter bleiben unberücksichtigt. In der letzten Zeile überschreibt der Store das Wort T[4] bei B + 16 mit 2.

Die Grenzwert-Variante ändert zwei Stellen:

```
    addi s0, x0, 0              # Anzahl = 0
    addi t3, x0, -1             # Grenzwert -1
Loop:
    bge  t0, t1, Done
    lw   t2, 0(s1)
    bge  t2, t3, Next           # nur Werte unter -1 zählen
```

Das sind Modellvorhersagen; Adressen, Anzeige und Ausgabe des eigenen Laufs gehören ins Protokoll.